

YAML

```
name: imagegen
description: "Use for visual deliverable routing and image generation/editing t
license: Complete terms in LICENSE.txt
```

ImageGen Manus

Use this skill to decide the correct production route for visual deliverables. A correct visual is not merely attractive; it must fit the user's intended medium, purpose, constraints, hierarchy, and acceptance criteria.

Core Principle

Identify the image's **job** before choosing a tool or style. Optimize for conversion, explanation, brand recognition, product accuracy, game usability, UI credibility, readable text, precise data mapping, or edit fidelity as appropriate.

Do not start by asking about model names, API parameters, or internal generation settings. Ask the user only for missing user-facing constraints that affect correctness, such as final medium, aspect ratio, exact text, language, brand colors, reference style, product constraints, transparency, or preservation requirements.

Authoritative Routing

Use the simplest route that satisfies the user's likely deliverable. Explicit user requirements override defaults. When multiple explicit deliverables are requested, produce them in the user's stated order; if no order is stated, start with the most directly named deliverable.

User intent	Required route
Explicit Mermaid , sequence diagram/时序图, flowchart/流程图, ER/class/state/Gantt diagram, graph syntax, dependency graph, hierarchy, formal architecture diagram, or any node-edge/process/system diagram where relationships must be accurate	Write Mermaid or another deterministic diagram source first. Render to PNG/SVG only if an image artifact is requested or useful for delivery. Do not replace a structured diagram request with AI-generated diagram art.
Numeric data, CSV/spreadsheet/table values, axes, scales, coordinates, measured comparisons, trend lines,	Use Python plotting or another deterministic visualization workflow. Preserve numeric mapping, units, axis ranges, tick labels, legends, and proportions. Use AI generation

statistical/scientific/financial charts, or “比例尺要准确/精细图表”	only for optional decorative assets, not for the chart itself.
Explicit code, 写代码, HTML/CSS/React, source files, demo/制作demo, working or functional prototype, interactivity, deployment, real website/app, or implementation	Build the requested software deliverable. Do not answer with image-only output. A bare “prototype/原型” request without code/demo/functionality wording is not automatically a development request.
UI image, app screen image, dashboard image, visual mockup, prototype image/原型图, 生成UI, poster, illustration, logo, icon, product visual, sticker/card/game asset, marketing visual, or other final visual asset	Use AI image generation or image editing unless the same request explicitly names Mermaid, precise charting, source code, a demo, or functional implementation. Deliver final image files.
Existing image plus requests to replace, remove, translate, localize, restyle, add objects, alter a region, preserve identity/product, change content, or upscale/restore/enhance resolution	Use image editing/generation. Preserve unchanged areas explicitly. Do not use deterministic image processing for creative or semantic edits. For upscale or restore requests, use the image variation tool with the exact prompt specified in the Upscale/Restore recipe in <code>references/prompt-recipes.md</code> ; do not add extra instructions or rephrase it.
Resize, crop, rotate, flip, format conversion, simple stitching, or compression where no new visual content needs to be created	Use deterministic image processing. When a resize or crop changes the aspect ratio, check whether the user accepts losing edge content. Signals that the user accepts content loss include words like "crop to", "trim", "cut off the edges", or "just the center part". When the user has not signaled acceptance of content loss, default to preserving all content: use AI image editing to reconstruct or extend the image so that no existing content is sacrificed.
Ambiguous “画个图/做个图/做个示意/帮我可视化/design a page/做个界面/做个原型”	Match conditions rather than blindly following a fixed output sequence: Mermaid for structural relationships, Python for numeric precision, AI generation for visual or conceptual assets, static layout/code screenshot for deterministic static composition, and web/app development only for explicit functional or source-code deliverables. Deliver the first suitable version, then offer one focused next step.

Diagram Boundary

Separate **structured diagrams** from **conceptual explanatory visuals**.

Type	Use when	Route
Structured diagram	Node-edge relationships, sequence, architecture dependencies, formal process flow, hierarchy, labels, or machine-readable structure must be accurate	Mermaid or deterministic rendering first
Conceptual explanatory visual	The user wants a polished presentation visual, framework image, educational visual, decorative schematic, or infographic where visual communication matters more than exact graph syntax	AI image generation after organizing labels and hierarchy
Both accuracy and polish matter	The user needs a correct structure and a visually polished version	Create the structured diagram first, then optionally create a polished visual version after the structure is approved

Scenario Classification

Assign one primary scenario and any secondary scenario before prompting or editing.

Scenario	Use when the visual is for	Success criterion
Website or landing page	Hero images, section visuals, blog covers, page visuals	Layout fit, whitespace, brand tone, text-safe area
Product or commerce	Product renders, packaging shots, marketplace images	Product accuracy, material credibility, clean presentation
Marketing or social	Ads, posters, campaign visuals, thumbnails	Strong focal point, emotional clarity, readable hierarchy
UI or app mockup	App screens, dashboards, interface concepts	Plausible layout, consistent controls, legible structure
Logo or icon	Brand marks, app icons, favicons, symbolic assets	Simplicity, recognizability, scalability

Game or asset pack	Sprites, props, tiles, characters, backgrounds	Reusable silhouette, consistent perspective, clean edges
Character or portrait	People, mascots, avatars, recurring characters	Identity consistency, natural anatomy, intended mood
Diagram or infographic	Explainers, conceptual visuals, structured visuals	Correct structure, readable labels, low ambiguity
Transparent asset	Cutouts, stickers, overlays, compositing assets	Clean alpha, complete subject, no unwanted background
Precise image edit	Modification of an existing image	Change only requested regions; preserve everything else
Image upscale or restore	Upscaling, restoring, or enhancing resolution of an existing image	Higher resolution and clarity while strictly preserving original content, style, and identity

Read `references/scene-decision-matrix.md` when scenario tradeoffs are unclear or when a request spans multiple media.

Cross-Cutting Policies

UI images

Default UI-related image requests to direct image generation as visual mockups. Do not create HTML/CSS/React, render a webpage, and screenshot it merely because the image resembles an interface. Use code, web/app development, or screenshot workflows only when the user explicitly asks for source code, editable front-end files, interactivity, deployment, a working prototype, or deterministic layout output.

When prompting UI images, specify screen type, navigation, information architecture, component hierarchy, realistic spacing, device or browser frame, light/dark mode, brand palette, and exact UI labels or sample data. Describe the output as a visual mockup; do not imply functional software.

Text-bearing visuals

Default text-bearing visual image requests to direct generation of the complete final image, with the text rendered by the image generation model itself. Treat Chinese, English, Japanese, Korean, Arabic, and other writing systems as first-class generation targets; do not assume non-English or dense copy requires a separate text overlay workflow. Text

presence, language, or amount of copy is never a reason to generate a blank/background-only image and then add text with Python, scripts, canvas, HTML, SVG, PIL, or manual compositing.

Strictly forbidden: do not create a poster, menu, infographic, UI mockup, card, flyer, banner, or other text-bearing image by first generating an empty background and then overlaying the required text programmatically. This prohibition applies especially to Chinese text and multilingual layouts. Use deterministic layout or post-processing only when the user explicitly asks for editable source/layout files, pixel-perfect corporate typography, print-production files, legally exact fine print, exact long tables, machine-readable diagrams, or source-controlled design assets; even then, do not present that route as a workaround for Chinese or non-English text generation.

Before prompting, organize meaningful copy into title, subtitle, sections, labels, callouts, CTA, footnotes, and UI labels as appropriate. Preserve exact proper nouns, prices, dates, numbers, Chinese characters, punctuation, and required terminology. Reduce redundant wording only when it harms visual readability and is not user-required.

Verify generated text against the user-stated requirements: required wording, language, hierarchy, placement, omissions, duplications, and critical numbers/dates/names. Do not run a separate generic garbled-text audit. If one or two targeted generations still fail critical text accuracy and exactness is required, regenerate with a narrower text block, reduce visual density, or ask the user whether exact editable layout/source files are required; do not fall back to Python/scripted text overlay as the default correction path.

Static layout/code screenshot

Use a static layout or code screenshot workflow only when the user explicitly needs deterministic static visual layout, exact typography or placement, reproducible image composition, editable layout/source files, or source-controlled design assets, but does not need functional software, deployment, data interactivity, or editable application source. Do not use this route merely because the visual resembles a UI, contains Chinese/non-English text, or contains a lot of text.

Multi-item visual sets

For card decks, icon packs, sticker packs, poster sets, product image sets, game asset packs, or similar requests, deliver **one standalone final image per requested item** by default. Do not deliver process images, raw subject-only illustrations, contact sheets, stitched previews, A4 sheets, PDFs, or combined boards unless explicitly requested. Archives may supplement many files, but must not replace direct access to final individual images when the user asked for images.

Image edits and references

For image edits, change only the target region/object/text and preserve identity, pose, product shape, background, lighting, perspective, color palette, and non-target objects unless the user requests otherwise. When the user says “use this as reference,” match only the relevant dimension: identity, style, composition, color, or product form.

Scenario-specific guardrails

Scenario	Guardrail
Logo/icon	Start with a simple symbol or mark. Do not rely on generated typography for precise brand wordmarks unless the user accepts concept-only output.
Product image	Preserve product shape, labels, proportions, materials, and legally sensitive marks. Do not invent branding unless requested.
Transparent asset	Require a true transparent background, complete subject, clean alpha, and no colored fringe or unwanted shadow.
Character	Define stable identity anchors: age, face shape, hair, outfit, silhouette, palette, and style.
Precise chart	Never use AI image generation for the quantitative plot itself.

Production Brief Workflow

Build a short internal brief before generating or editing.

Brief field	Determine
Purpose	What the visual must accomplish
Medium	Website, app, ad, marketplace, slide, game, print, social, or asset library
Subject	Main object, person, environment, interface, product, or concept

Composition	Orientation, framing, whitespace, safe area, hierarchy, camera angle, screen/device context
Style	Photographic, vector, 3D, editorial, flat, pixel art, brand style, or reference-matched
Text/content	Exact wording, language, hierarchy, labels, UI copy, callouts, or “no text”
Constraints	Aspect ratio, dimensions, transparency, brand colors, preservation requirements, excluded objects
Acceptance risks	Errors that would make the result unusable

Ask only for missing essential information. If nonessential details are unspecified, make a reasonable assumption and proceed.

Use this prompt structure for new visuals:

Plain Text

Create [image type] for [use case and audience].
 Subject: [specific subject, interface, product, scene, or concept].
 Composition: [framing, orientation, focal point, safe area, background depth, style].
 Style: [visual style, lighting, material, color palette, brand tone].
 Text/content to render: [organized exact content block, UI labels, section labels].
 Constraints: [aspect ratio, transparency, text handling, elements to include/exclude].
 Avoid: [scenario-specific failure modes].

Use this prompt structure for edits:

Plain Text

Edit the provided image. Change only [target region/object/text].
 Preserve [identity, pose, product shape, background, lighting, perspective, color].
 New result should [desired outcome].
 Text/content to render if relevant: [exact replacement text and location].
 Avoid: [unwanted changes, artifacts, text errors, style drift].

Read [references/prompt-recipes.md](#) for scenario-specific prompt structures.

Lightweight Validation and Delivery

Do a **lightweight pass/fail check** before delivery, not an open-ended audit. Confirm only that the result satisfies the user's explicit request, the selected route is correct, and there is no obvious fatal defect that would make the artifact unusable. Do not perform forensic inspection, crop local regions for preview, run scripts to locate labels, search fonts, create masks, compare tiny details, or repeatedly re-open intermediate files unless the user requested that workflow or a visible critical failure blocks delivery.

If a critical requirement fails, correct only the single most important failure with one focused edit/regeneration or an appropriate route change. Do not keep refining for subjective polish, minor crop preferences, tiny spacing issues, uncertain text micro-errors, or speculative improvements. For text-bearing visuals, a quick visual check of requested headline/labels/numbers is enough; do not run a generic garbled-text audit or build a scripted correction pipeline.

Use deterministic image processing for operations where no new visual content is needed: downscaling, trimming, format conversion, or compression. When a resize or crop implies a new aspect ratio, default to preserving all content unless the user explicitly accepts edge loss; use AI editing to reconstruct or adapt the image when content must not be lost.

Deliver the artifact that matches the selected route: Mermaid source for Mermaid requests, chart image and source data/code as appropriate for Python plots, implementation files for code/demo/web/app requests, static layout images/source when requested, and final image files for AI generation or editing. Once the requested artifact has been delivered, **stop**. Do not continue generating extra variants, backup versions, additional fixes, or follow-up improvements after final delivery unless the user asks for them. Briefly state only essential scenario assumptions or known limitations that affect use.